



Data Warehouses

Lecture 11

Lecture Goals

Some notes on doing ETL in SSIS

General proces

Staging tables

General structure

Basic sanity checks

Cleaning data

Enriching data

Incremental load

The ETL process is a coordinated effort between numerous tools and utilities that are configured and assembled to load the star schema as required by its design documentation.

- Packaged software (ETL tools)

- “Hand-coded” programs

Different data warehouse architectures organize the task of data movement in different ways

- all require that information from source systems eventually make its way into a star schema

The process of loading a star schema involves two major categories of activity:

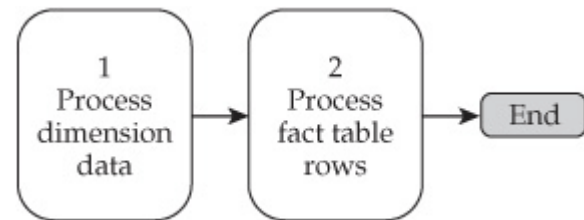
- processing data for dimension tables

- processing data for fact tables.

Understanding how a star schema is loaded begins with recognizing a fundamental dependency:

- each fact table row contains foreign key references to dimension table rows.

- implies that before a row can be inserted into a fact table, the appropriate dimension rows must be ready.



Loading a dimension table is an incremental process

- not a one-time-only affair

- it is complex by the need to process slowly changing dimensions - the load must identify new and changed data, manage surrogate keys, and insert or update dimension records as appropriate.

- It is necessary to inspect the data sources of dimension tables for new and changed information on a regular basis and insert or update records as appropriate.

Loading a transaction-grained fact table is comparatively simple

- The major challenge for this process involves replacing the natural keys that describe business transactions with surrogate keys that refer to the dimension tables.

What a Dimension Load Must Achieve

The process of loading a dimension table must incorporate the following major requirements:

- Extract source data from source(s)

- Assemble the dimensional attributes

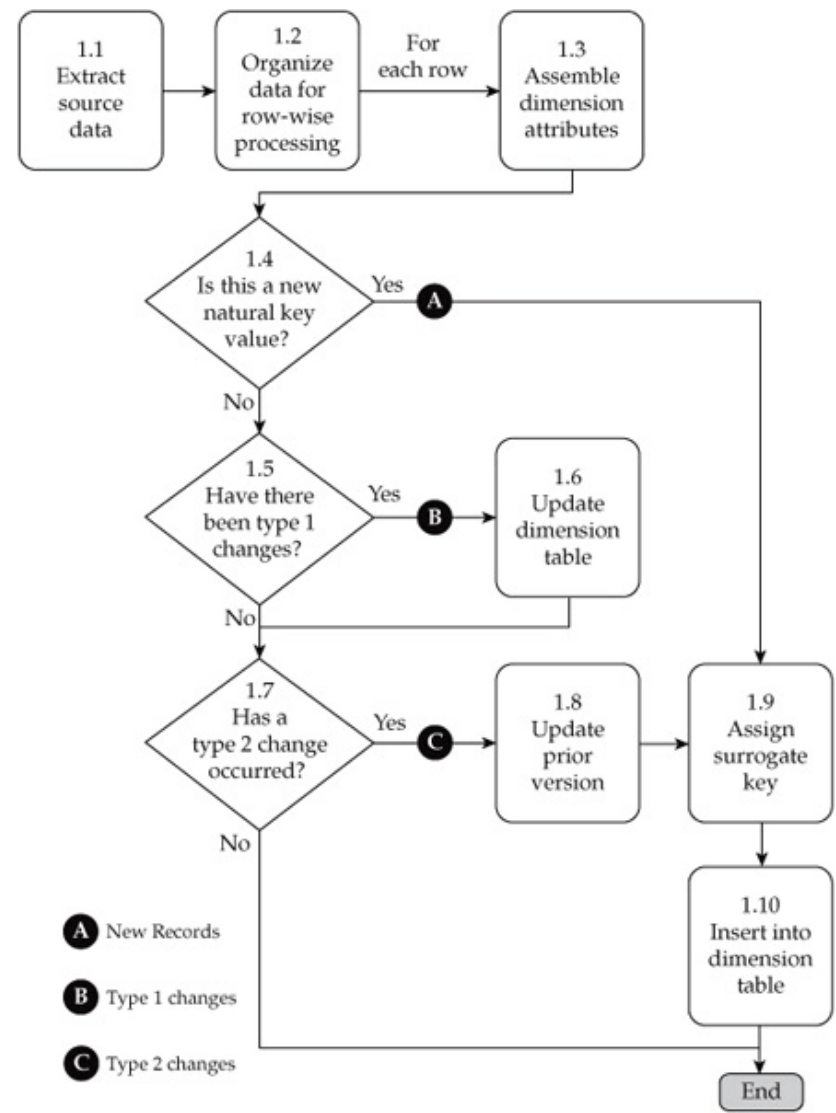
- Identify new and changed dimension records

- Manage surrogate keys

- Process new records

- Process type 1 changes

- Process type 2 changes



Design of PRODUCT dimension table:

PRODUCT	
product_key	SK
sku	NK
product_name	1
brand_code	2
brand_manager	2
.	.
.	.
current_version	HK

Legend:

SK Surrogate Key
 NK Natural Key
 1 Type 1 Change (Overwrite)
 2 Type 2 Change (Insert new record)
 HK Housekeeping Column (Internal use)

Existing content of PRODUCT Dimension Table:

SK	NK	1	2	2	HK
product_key	sku	product_name	brand_code	brand_manager	current_version
22	A-0033	Flat Box 12"	110	Lei, Michelle	Current
344	A-1011	Packing Tape	221	Jones, Paul	Current
1001	B-3691	Twine Spool	501	Smith, Dei	Not Current
1201	B-3691	Twine Spool	501	Klein, Pete	Not Current
2700	B-3691	Twine Spool	702	Jones, Fay	Current
2711	Z-3320	Envelope A5	109	Harvey, Ned	Current

Source records to be processed:

	sku	product_name	brand_code	brand_manager	
1	S-8472	Envelope B4	109	Harvey, Ned	A new record
2	B-3691	TwineSpool 12	702	Jones, Fay	Type 1 change affecting 3 dimension rows
3	A-0033	Flat Box 12"	110	Cook, Dan	Type 2 change
4	A-1011	Clear Packing Tape	221	Davis, Jon	
5	Z-3320	Envelope A5	109	Harvey, Ned	Type 1 and type 2 changes
					An unchanged record

The fact table load process takes transactions from the source system, calculates facts, identifies the appropriate foreign keys, and inserts records into the star.

The most complex part of this process involves identification of the appropriate foreign key values.

Source systems supply only a natural key, but the fact table must use foreign keys to refer to the dimensions.

For each fact in a source system, the natural keys that identify dimensional context must be replaced by surrogate keys that identify rows in dimension tables.

Because type 2 changes are possible, there may be multiple rows in the dimension table for a particular value.

What a Fact Table Load Must Achieve

The fact table load is governed by the following requirements:

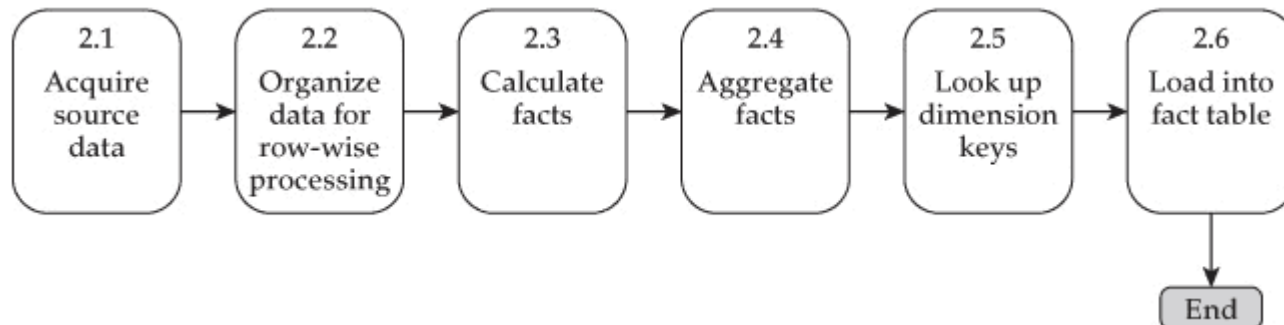
- Extract data from the source systems

- Compute the facts

- Aggregate facts to match the grain of the fact table

- Obtain surrogate keys for each of the dimensions

- Load fact table records into the warehouse



Orders being processed for FACT load:

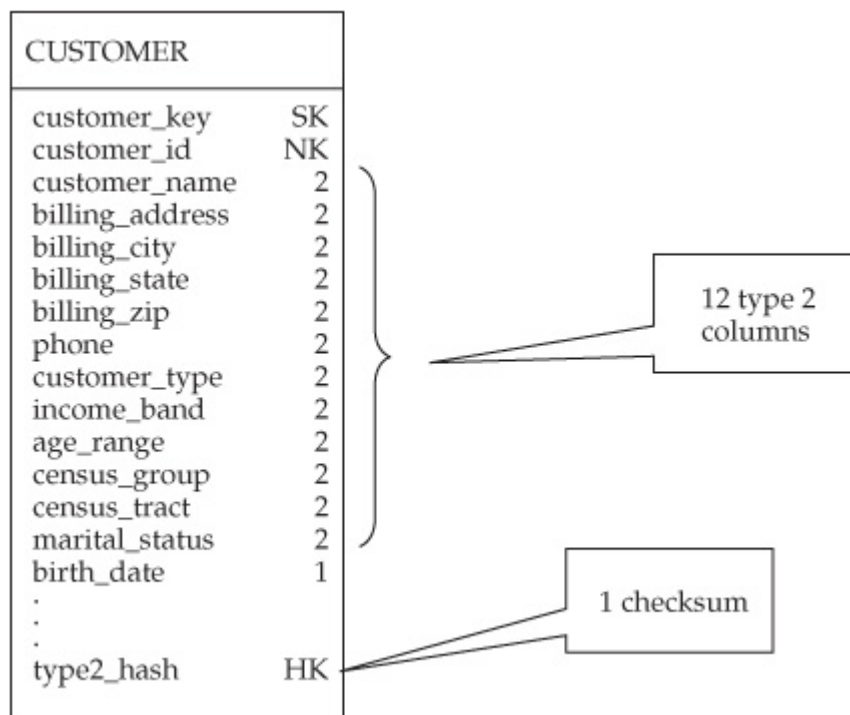
	date	order_ dollars	margin_ dollars	customer_ id	salesrep_ id	sku	product_ key
1	3/12/2010	1,700.22	643.06	2201	1197	S-8472	2799
2	3/12/2010	984.31	201.64	3832	1197	B-3691	

PRODUCT Dimension Table:

SK	NK	1	2	2	HK
product_ key	sku	product_ name	brand_ code	brand_ manager	current_ version
22	A-0033	Flat Box 12"	110	Lei, Michelle	Not Current
1192	A-0033	Flat Box 12"	110	Cook, Dan	Current
344	A-1011	Clear Packing Tape	221	Jones, Paul	Not Current
2828	A-1011	Clear Packing Tape	221	Davis, Jon	Current
1001	B-3691	TwineSpool 12	501	Smith, Dei	Not Current
1201	B-3691	TwineSpool 12	501	Klein, Pete	Not Current
2700	B-3691	TwineSpool 12	702	Jones, Fay	Current
2799	S-8472	Envelope B4	109	Harvey, Ned	Current
3001	Z-3320	Envelope A5	109	Harvey, Ned	Current

A

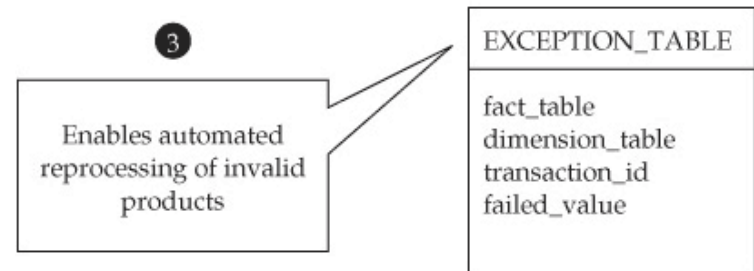
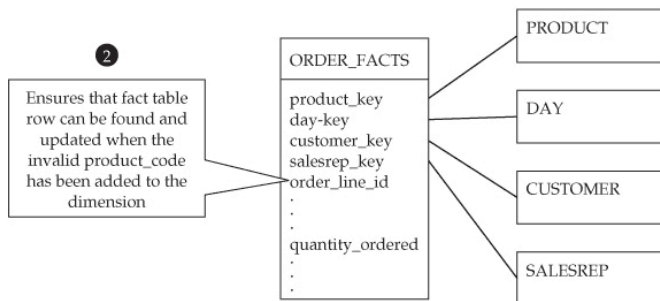
B



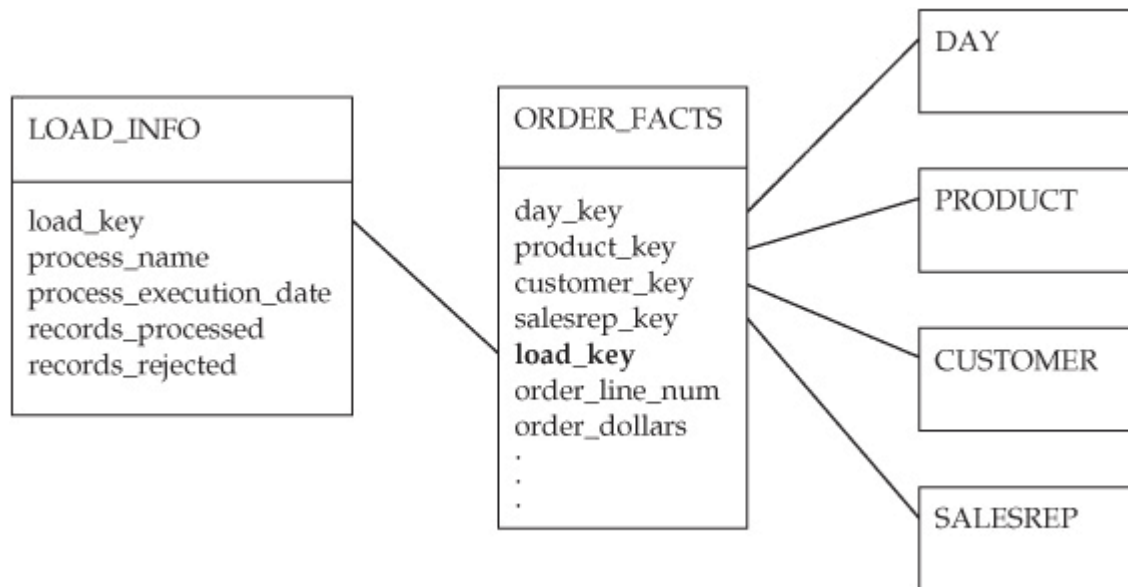
1

Used in the fact table when an order has an invalid product_code

PRODUCT				
product_key	row_type	product_code	product_name	
0	Invalid	[Invalid]	n/a	
101	Product	B57330-1	Cardboard Box	
102	Product	B47770-2	Bubble Envelope	



CUSTOMER		
customer_key	SK	
customer_id	NK	
customer_name	2	
billing_address	2	
billing_city	2	
billing_state	2	
billing_zip	2	
phone	2	
customer_type	2	
income_band	2	
age_range	2	
census_group	2	
census_tract	2	
marital_status	2	
birth_date	1	
⋮		
⋮		
current_version	HK	} Housekeeping Columns
effective_date	HK	
expiration_date	HK	
type2_hash	HK	
type1_hash	HK	
creation_date	HK	
update_date	HK	
update_process	HK	



Staging tables

Using ETL Staging Tables

When the volume or granularity of the transformation process causes ETL processes to perform poorly, consider using a staging table on the destination database as a vehicle for processing interim data results.

Staging tables are normally considered volatile tables, meaning that they are emptied and reloaded each time without persisting the results from one execution to the next.

Staging tables should be used only for interim results and not for permanent storage.

Staging tables

When using a load design with staging tables, the ETL flow looks something more like this:

- Delete existing data in the staging table(s)

- Extract the data from the source

- Load this source data into the staging table(s)

- Perform relational updates to cleanse or apply business rules to the data, repeating this transformation stage as necessary

 - (typically using T-SQL, PL/SQL, or other language specific to your RDBMS)*

- Load the transformed data from the staging table(s) into the final destination table(s)

Staging tables

By loading the data first into staging tables

You are able to use the database engine for things that it already does well

For example, joining two sets of data together for validation or lookup purposes can be done in most every ETL tool

but this is the type of task that the database engine does exceptionally well.

Same thing with performing sort and aggregation operations; ETL tools can do these things,

but in most cases, the database engine does them too, but much faster.

Staging tables

Options for Staging Tables

When using staging tables to triage data, you enable RDBMS behaviors that are likely unavailable in the conventional ETL transformation.

For example, you can create indexes on staging tables to improve the performance of the subsequent load into the permanent tables.

You can run multiple transformations on the same set of data without persisting it in memory for the duration of those transformations which may reduce some of the performance impact.

Staging tables also allow you to interrogate those interim results easily with a simple SQL query.

Further, you may be able to reuse some of the staged data, in cases where relatively static data is used multiple times in the same load or across several load processes.

General structure

How to package?

Building the Extract, Transform and Load logic separately

created multiple packages per single dimension/fact table:

One extract package where the extract(E) logic of all dim/fact tables is stored

One dim/fact package with the transform(T) logic of a single dim/fact table

One dim/fact package with the load(L) logic of a single dim/fact table

multiple reasons:

Enable running the E/T/L parts separately, for example: run only the entire T phase of all dim/fact tables.

Run the extracts of all dimensions and fact tables simultaneously to keep the loading window on the source system as short as possible.

General structure

How to package?

Idea of having one package per dimension / fact table, with complete E/T/L logic, for the following reasons:

All the logic is in one place

You can see your packages as separate ETL “puzzle pieces” that are reusable

Increase understandability

Perform unit testing

If there is an issue with a dimension or fact table, you only have to make changes in one place, which is safer and more efficient

Simplifies deployment after changes have been made

Changes are easier to track in source control systems

Example

parent-child package structure:

One parent (master) package will execute multiple child (dim/fact) packages.

This solution is based on a single (child) package for each dimension and fact table.

General structure

How to package?

One Package = One Unit of Work

break apart logical units of work into separate packages in which each package does just one thing

a few design patterns that work well when populating a data warehouse:

- Each dimension has a separate package.

- Each fact table has a separate package.

- Staging operations (if used) each have their own package.

- Functional operations unrelated to data movement (for example, dropping or disabling indexes on tables before loading them with data) are separated as well.

- You can group some of these operations together in common packages where appropriate;

- for example, if you truncate tables and drop indexes in a staging database, those operations typically reside in the same package.

Basic sanity checks

You can use simple row counting to investigate if all rows are present in a particular table

In SSIS row counting can be performed

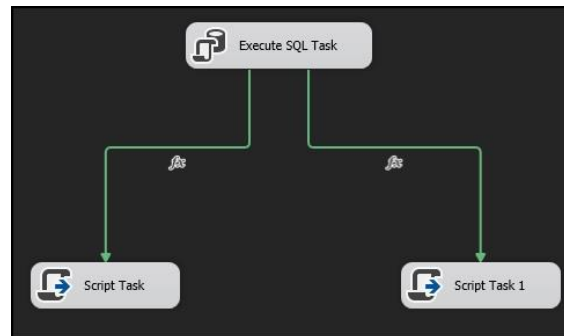
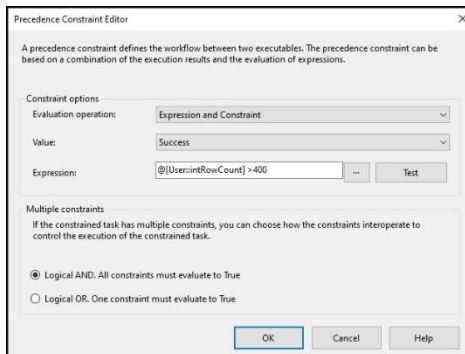
Using Execute SQL Task – by running a proper SQL query and assinging its result to a variable

Using a Row Count task in Data Flow – by letting the system to count the number of flowing rows and assing it to a variable

Further you can define precedence relations in Control Flow

Define an expression that is evaluated (compare row counts) to determine whether the constrained executable runs.

If the expression evaluates to true, the constrained executable runs.



Basic sanity checks

“Into each life some rain must fall.”

The world of ETL is no different, except that rain comes in the form of errors, often as a result of missing or invalid data

The good news is that there are data warehousing design patterns that can help mitigate the risk of data anomalies that interrupt the execution of Integration Services packages.

Basic sanity checks

You can use aggregate totals (which might include financial amounts or other summary data) in a particular table

In SSIS aggregation can be performed

Using Execute SQL Task – by running a proper SQL query and assigning its result to a variable

Using an Aggregate task in Data Flow – by letting the system to aggregate the data based on provided conditions and later assigning the result to a variable

Similar to the previous example, you can further define precedence relations in Control Flow that uses the introduced variables

Compare them

Basic sanity checks

Don't forget to check for cases where no data is loaded.

This happens more often than you might think.

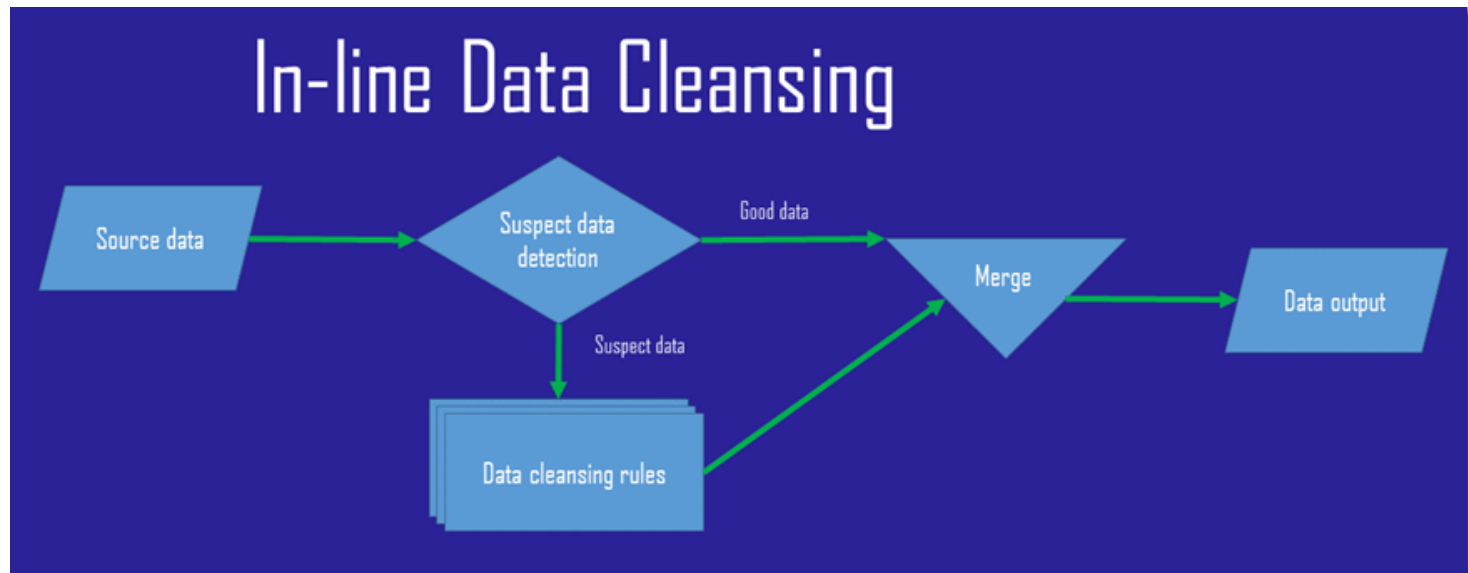
A source data file containing no data, an improperly configured query that returns no rows, or an empty source directory intended to contain one or more files could all lead to the ETL process running successfully but loading exactly zero rows of data.

If a given process should always result in a nonzero number of rows loaded, be sure to add an audit step to verify this

Cleaning data

When bad data is found, use the ETL process to clean up and continue processing that data if possible

Resolving data deficiencies in-flight

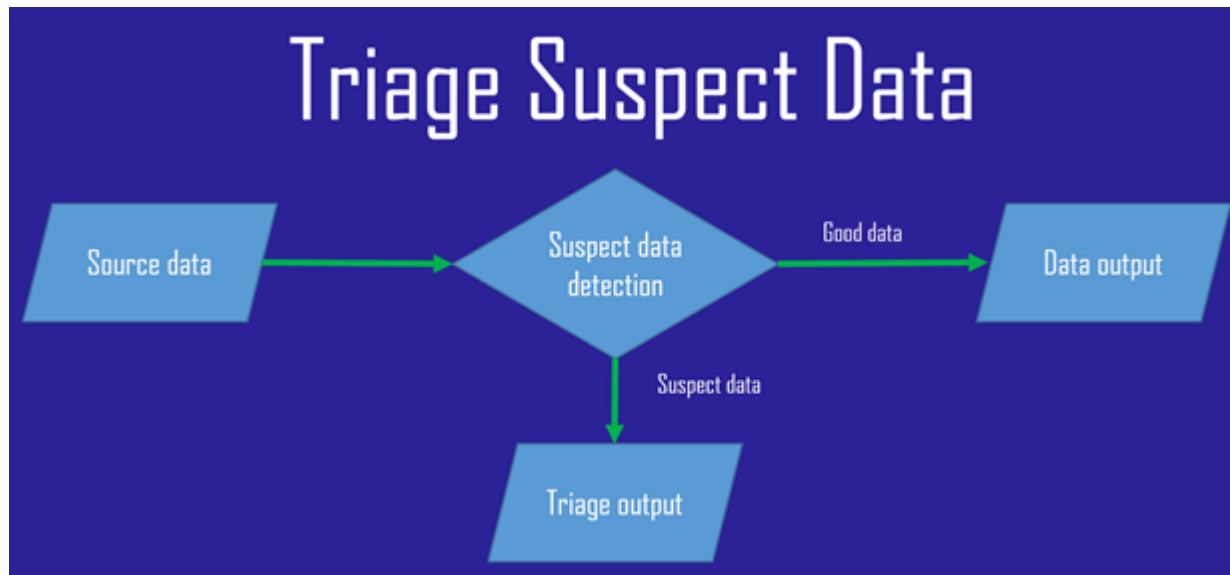


Cleaning data

The triage design pattern is useful for data quality issues that cannot be resolved in-line

- the suspect rows are left intact and are written to an alternate location to be dealt with in a separate process

- in most cases, there are additional processes – or perhaps even human intervention – for remediating triaged data and loading it to the final destination



Cleaning data

A basic approach to clean data

- Using predefined dictionaries

- Using predefined patterns

Cleaning data

Using predefined dictionaries

Source

You can manually create a list of correct values (or known incorrect ones)

Like customer names

You can use publically available lists of correct values

Like geographical names

Usage

Include Lookup Transform in Data Tools

Dictionaries are typically small and can be loaded into memory for a quick lookup operation

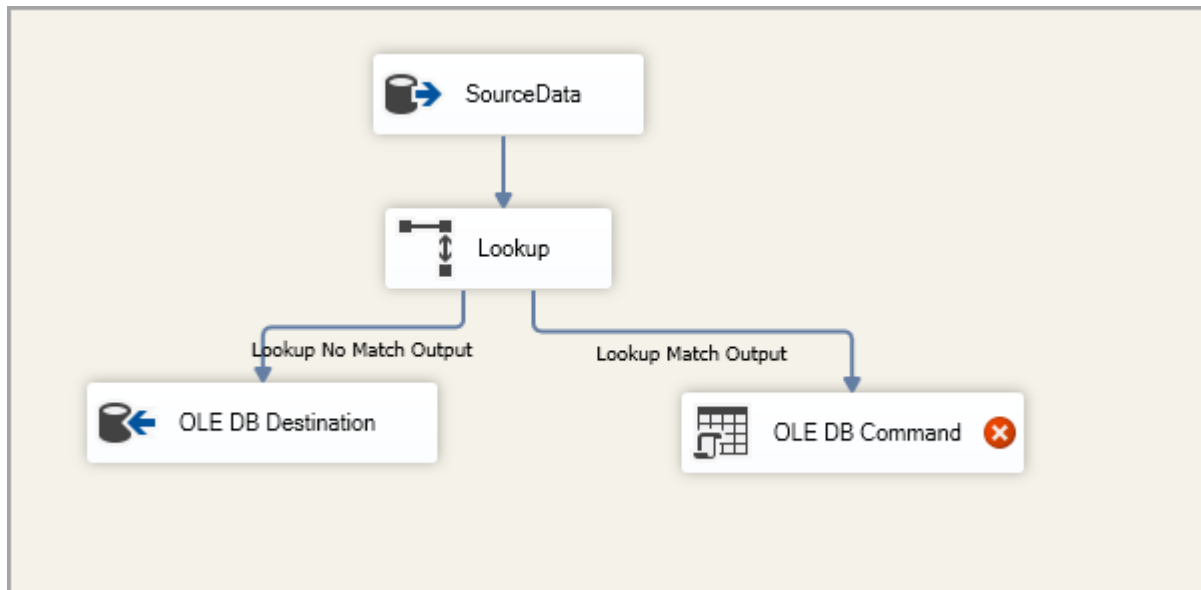
By default, the Lookup Transformation uses the OLE DB Connection Manager for the lookup reference table

As an alternative, if the dictionaries are available as external files, you can load them into memory using Cache Transformation (Data Flow)

Cleaning data

Using Lookup Transformation

Remember to handle the not matched rows



Cleaning data

Using predefined patterns

Some attributes should have a well defined pattern

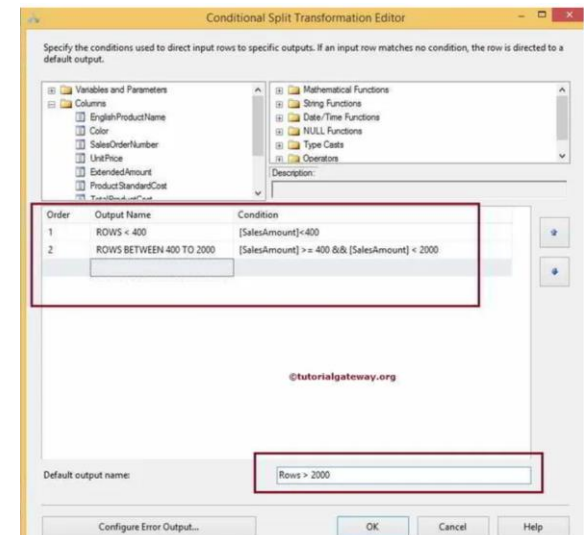
E.g. e-mail, postcode

You can use a Conditional Split Transformation in SSIS to test for the pattern

Conditional Split is just like the IF condition or CASE statement.

It checks the given condition and based on the condition result can output the row to an appropriate destination path.

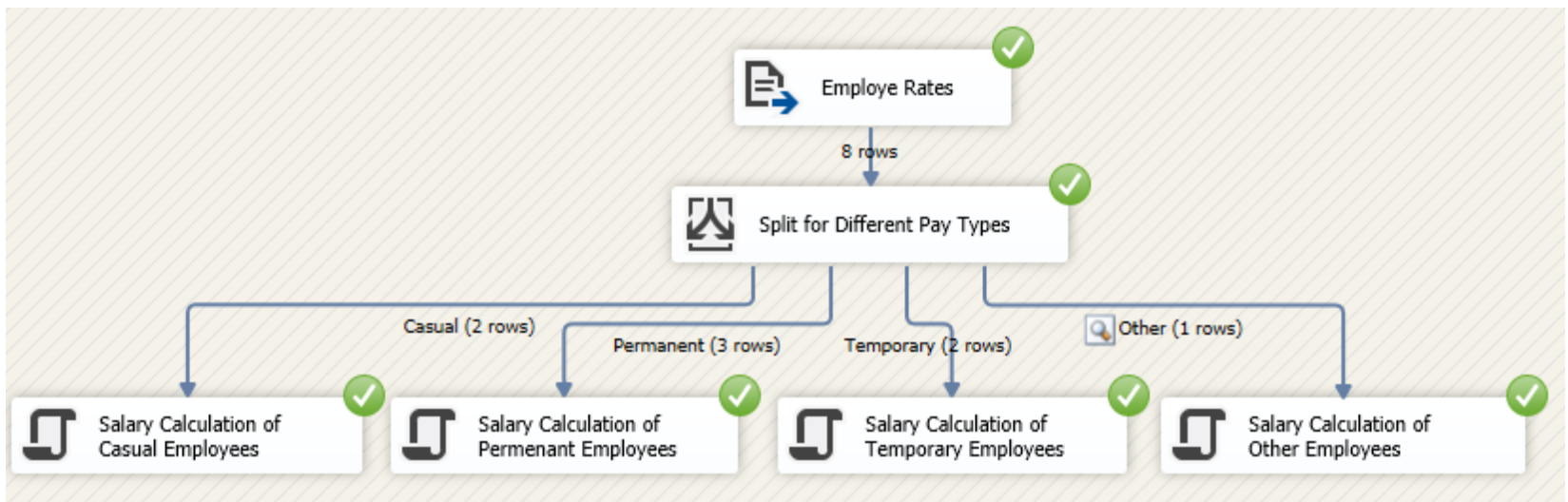
It has ONE input and MANY outputs.



Cleaning data

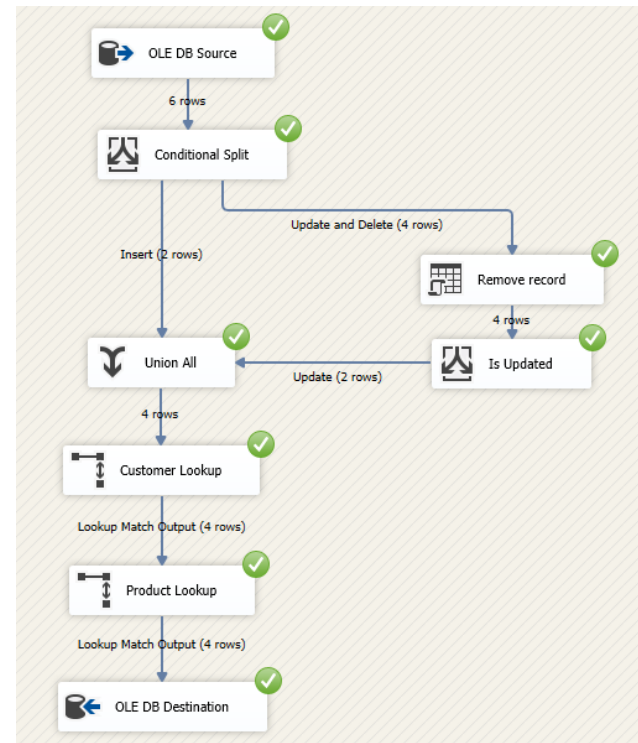
You can later capture each output and assign it a new flow and apply adequate transformation

E.g. clean the incorrect postcode based on external dictionary



Cleaning data

Moreover, you can later use Union All transformation
Union the correct and cleaned rows together

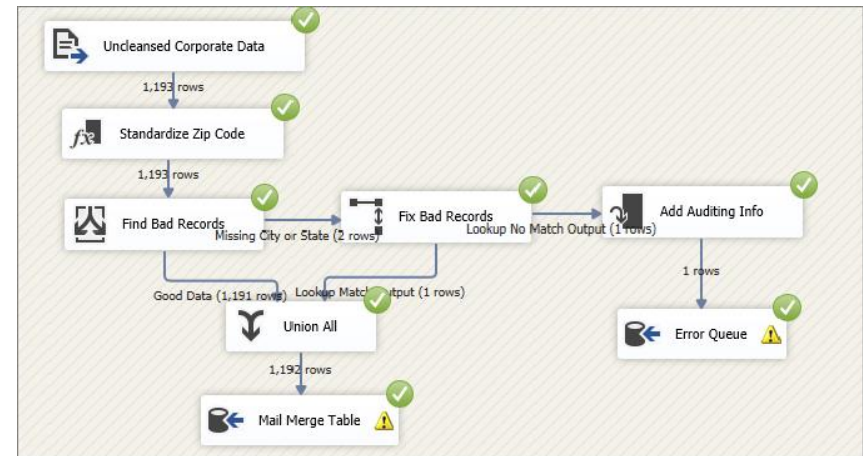


Cleaning data

In a more complete solution should also add auditing info

In SSIS you can use the Audit Transformation

Add additional columns to the output.



Enriching data

To perform some basic operations on rows you can use Derived Column transformation

Derived Column Transformation in SSIS plays a vital role in dealing with expressions in SQL Server Integration Services.

The Expression Language has powerful built-in functions for string manipulation, data type conversions, mathematical functions, conditional expressions, and handling Null values.

The SSIS Derived Column Transformation also provides an option to choose whether you want to create new columns or replace existing ones with values calculated from expressions.

It has many basic uses, from basic replacing of NULLs or blanks to text parsing and manipulation.

Enriching data

The Derived Column

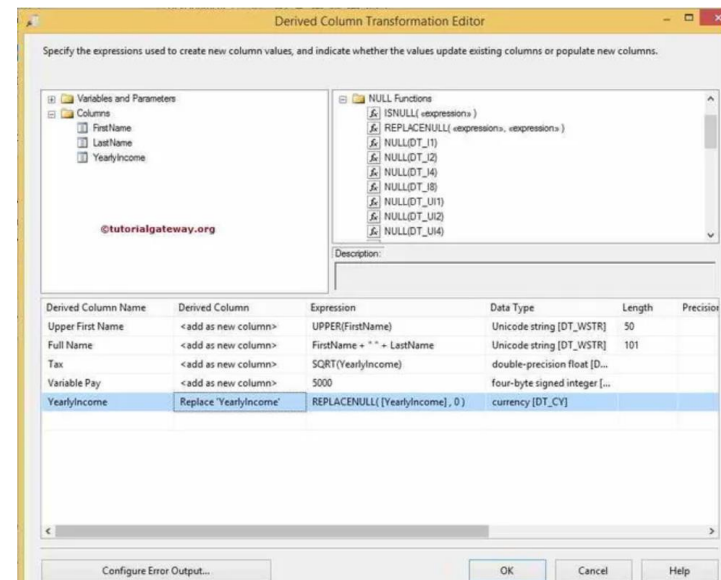
Transformation can be used for more advanced data cleansing operations, such as the following:

Advanced text code logic to identify and parse text values

Checking for data ranges and returning a specified value

Mathematical operations with advanced logic

Date comparison and operations



Missing Data in Dimensions

Missing Data in Dimensions

Prevent by proper loading order

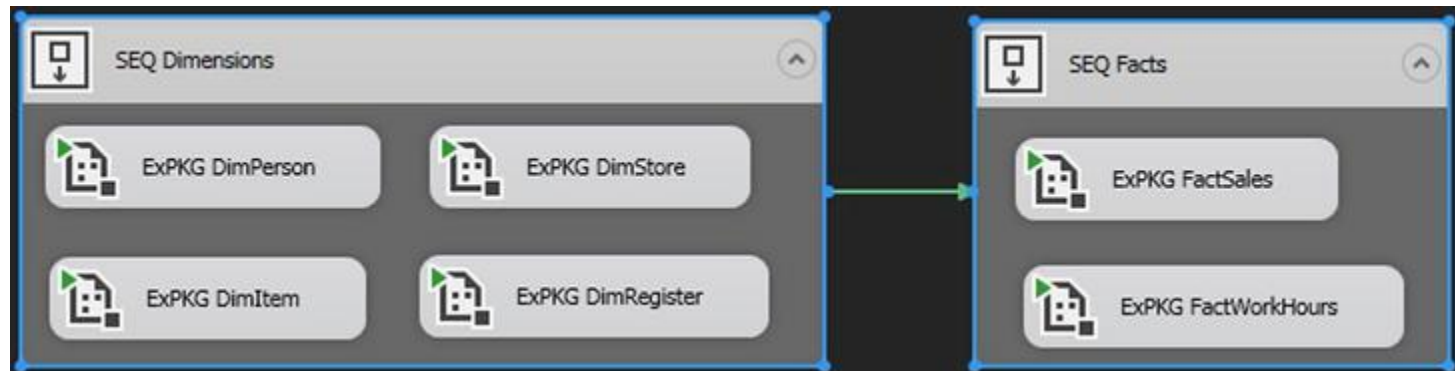
Use the unknown member

Loading

With respect to data warehousing, a more common example of handling errors inline is the case of late-arriving dimension data.

The typical pattern is to load the dimensions first, followed by a load of the fact tables.

This helps to ensure that the fact records will find a valid dimension key when the former is loaded to the data warehouse.

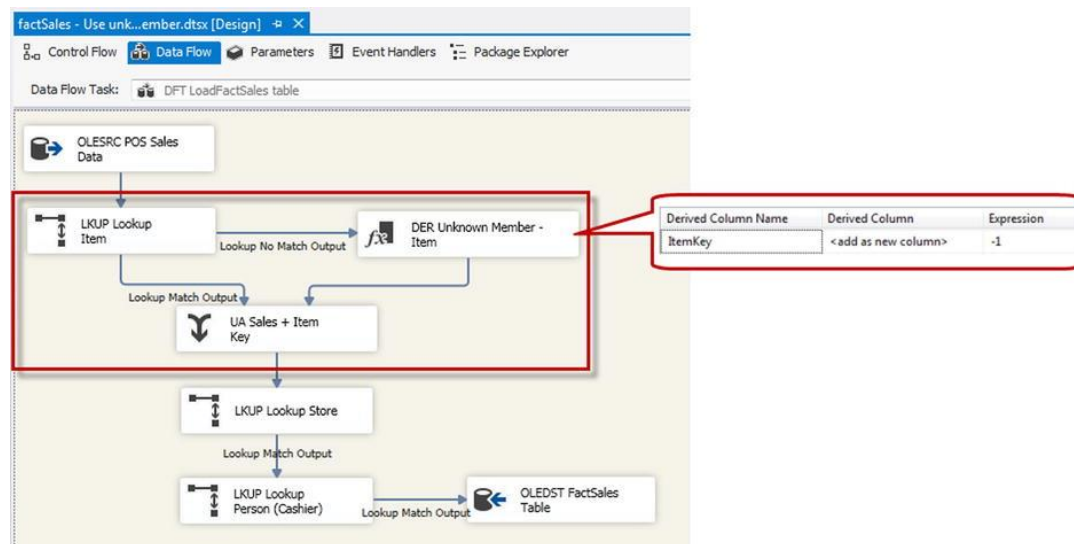


Unknown member

Using the unknown member

This pattern generally works best when the fact record alone does not contain the required information to create a unique new dimension record.

all of the unknowns would, by default, be grouped together for analytical purposes



Add the Missing Dimension Member

Add the Missing Dimension Member

Using this design pattern, you can add missing dimension records on the fly as part of the fact package using as much dimension data as is provided by the fact data.

In this scenario, the fact records are immediately available in the data warehouse, just like they were for the previous design pattern, but this methodology has the added benefit of matching the fact record to its proper dimension member.

In most cases, this allows you to immediately associate the correct dimension data rather than group the unmatched data into the unknown member bucket.

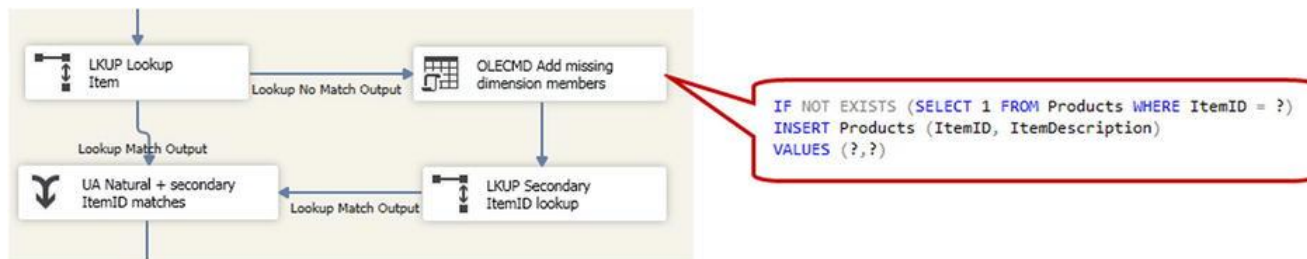
It can be reasonably assumed that the incoming fact data will not completely describe the dimension member

For that reason, you should also complement this design pattern with a process that attempts to fill in the missing dimension elements (which may already be addressed as part of a comprehensive slowly changing dimension strategy).

Add the Missing Dimension Member

Instead of simply assigning the value of the unknown member using the Derived Column transform, we leverage an instance of the OLE DB Command transform to insert the data for the missing dimension record in the fact table into the dimension table.

The SQL statement is shown in the callout, and in the properties of the OLE DB command, we map the placeholders (indicated by question marks) to the appropriate values from the fact record.

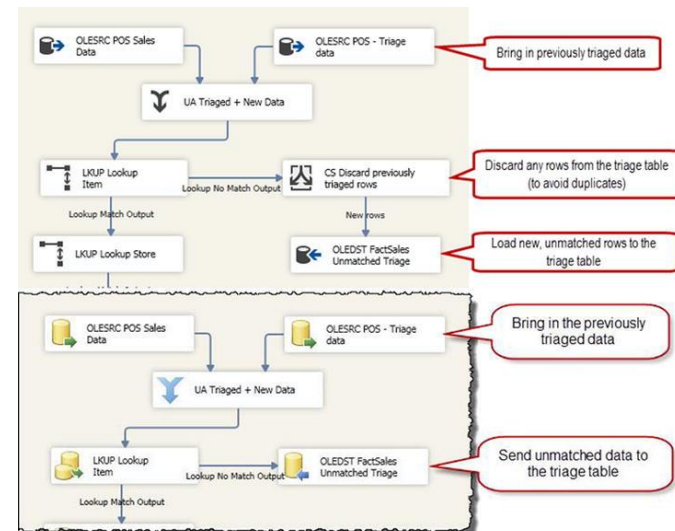


Triage the Lookup Failures

For missing dimension records, the most common approaches typically involve one of the preceding.

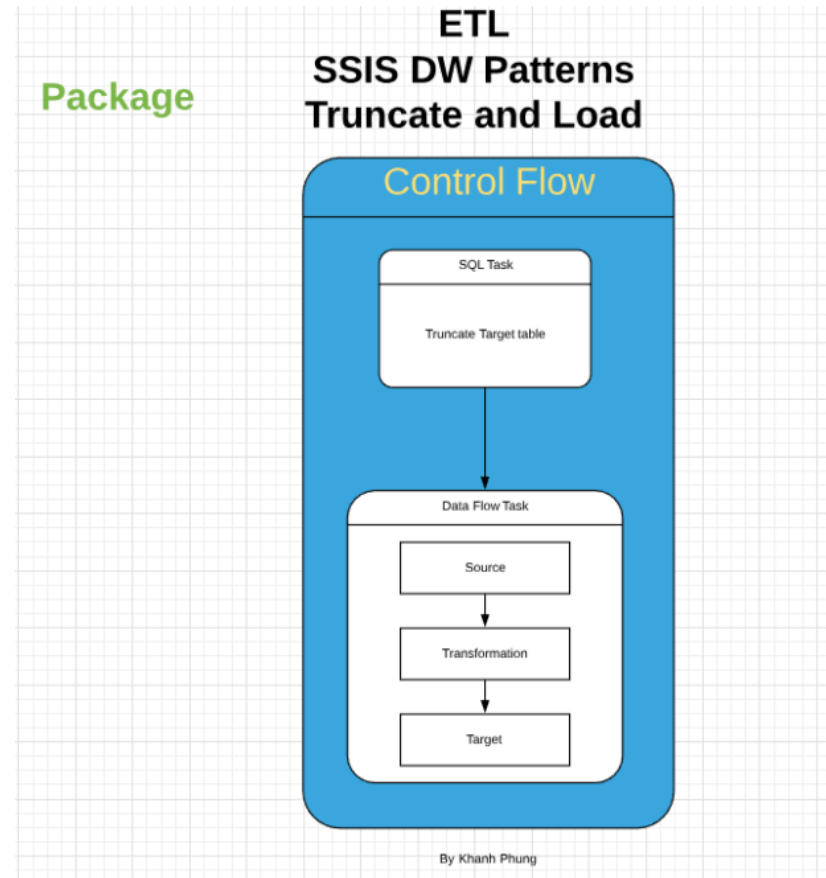
However, on occasion it will be necessary to delay the processing of fact records that do not match an existing dimension record.

In cases such as this, you'll need to create a triage table that will store the interim records until they can be successfully matched to the proper dimension.



Full refresh in SSIS

Is an option, not the option



Incremental Loads in SSIS

Using the Control Table pattern for incrementally loading data

Use a table to determine when the last load of the data was, then the package will determine which rows to load based on the last load date.

The pattern uses a table that the developer creates to store operational data about the last load

a row in the control table for each table that you wish to create a load process for

becomes also an audit table to see which tables have and have not been loaded

Incremental Loads in SSIS

A simple incremental load patterns follows three steps:

An Execute SQL Task reads the last load data from the control table into a variable.

A Data Flow Task reads from the source table where they were modified or created after the date in the variable.

An Execute SQL Task sets the last load date in the control table to the time when the package began.

```
CREATE TABLE [dbo].[ControlTable](
    [SourceTable] [varchar](50) NOT NULL,
    [LastLoadID] [int] NOT NULL,
    [LastLoadDate] [datetime] NOT NULL,
    [RowsInserted] [int] NOT NULL,
    CONSTRAINT [PK_ControlTable] PRIMARY KEY CLUSTERED
(
    [SourceTable] ASC
)
) ON [PRIMARY]
```

Incremental Loads in SSIS

A simple query to find out the last time you retrieved data from the table called SourceTable.

```
SELECT LastLoadDate from  
ControlTable where SourceTable = 'SourceTable'
```

The answer to the query should be stored into a variable to be used later.

To define a result of a SQL query to a variable in SSIS

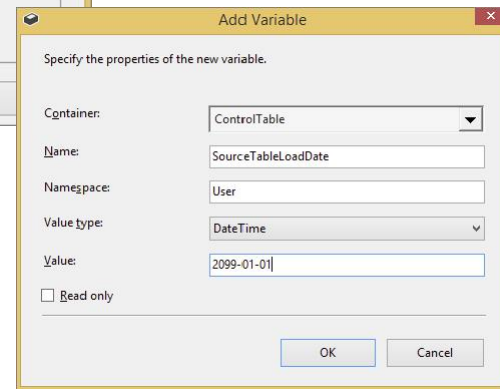
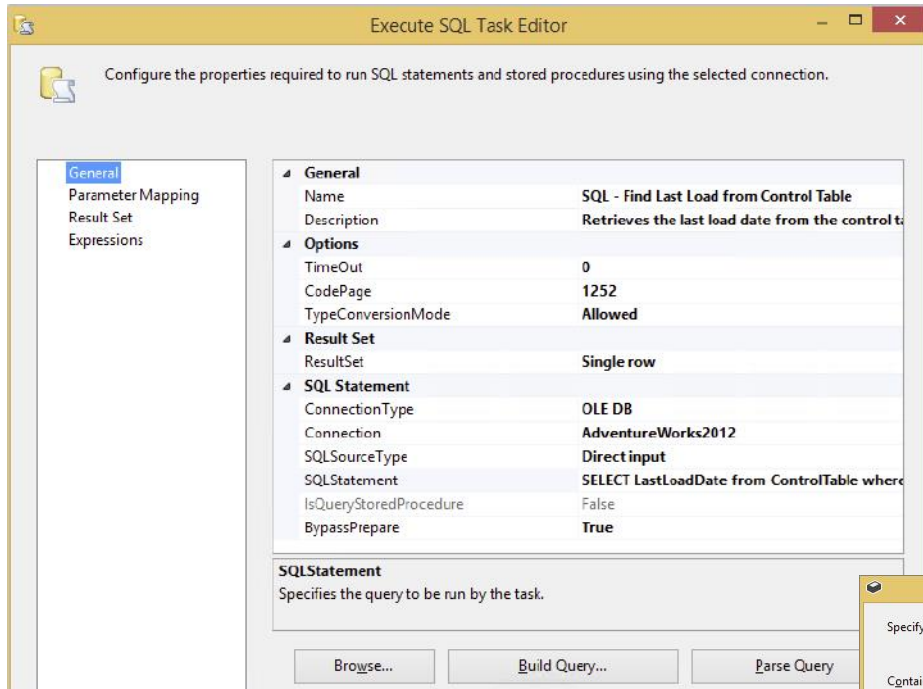
- Create a Execute SQL Task

- Change ResultSet to Single row

- In the Result Set tab, add a new resultset – change the name to 0 (first column) and assign it a variable (or create a new one)

- When creating a new variable try assigning some future absurd date, like 2099-01-01, to make sure that without running the query this variable won't affect the load*

Incremental Loads in SSIS



Incremental Loads in SSIS

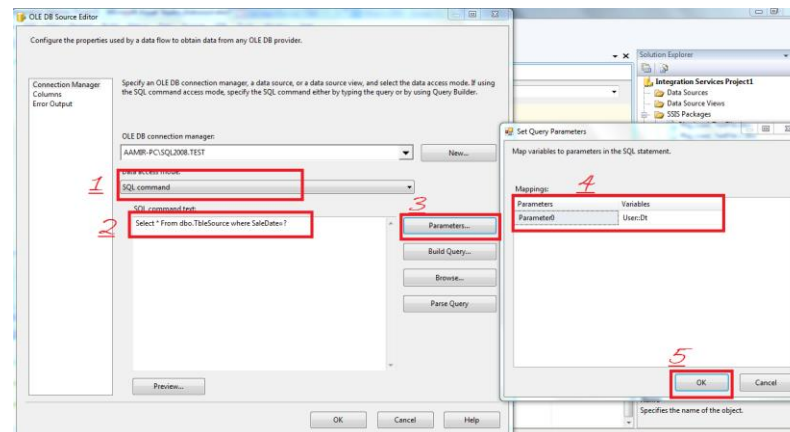
Having the variable in place you can now query the Source Table

Using an OLE DB Source in a Data Flow

As Data Access Mode choose SQL Command

```
SELECT * from SourceTable WHERE CreatedDate BETWEEN ? and ?
```

Set Parametr using the variable and System::StartTime (current date)



Incremental Loads in SSIS

Obviously, after such a load, the ControlTable should be updated

You can use Execute SQL Query task

```
UPDATE ControlTable  
SET LastLoadDate = ?  
WHERE SourceTable = 'SourceTable'
```

With parameters – current date

Configure the properties required to run SQL statements and stored procedures using the selected connection.

General	Variable Name	Direction	Data Type	Parameter Name	Parameter ...
Parameter Mapping	System::StartTime	Input	DATE	0	-1
Result Set					
Expressions					

Incremental Loads in SSIS

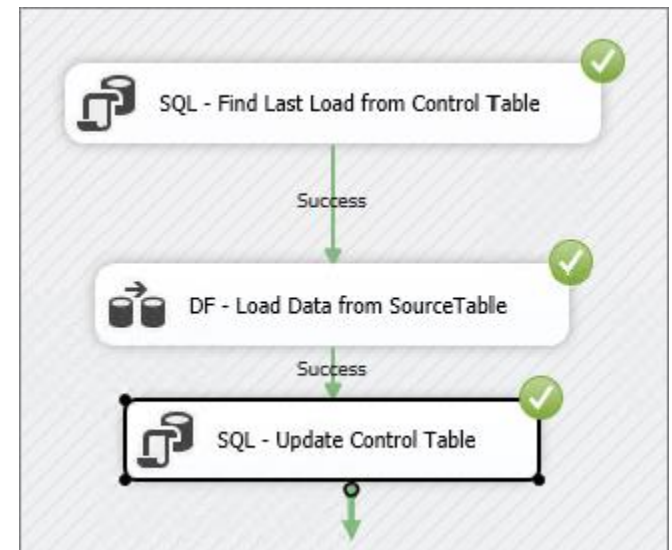
As a result you should have all of the rows created after the previous load of data

It is then up to you, to apply proper logic of how to treat this data

Identify changes / identify new rows

Ignore them / clean and conform them

Update / insert them



Incremental Loads in SSIS

What if there is no timestamp

Idea:

Comparing the target table against the source data

You can use native SSIS components and apply proper logic for checking for new data

Through the use of lookups, conditional splits, and OLE DB command components, you can create a simple data path through which we can process new and changed data from our source system(s).

Delta approach

Employing this design pattern is one of the most common ways to perform an incremental load using SSIS.

When properly configured, it can perform reasonably well.

This is also the design pattern with the fewest external dependencies - almost any data can be used as a source, it does not require database engine features

Incremental Loads in SSIS

When you are using this design pattern, the most common operations you will perform will include the following steps:

Extract data from the data source(s).

If more than one source is used, they can be brought together using the appropriate junction component (Merge, Merge Join, or Union All transforms).

Using the Lookup transform, join the source data with the target table based on the business key(s).

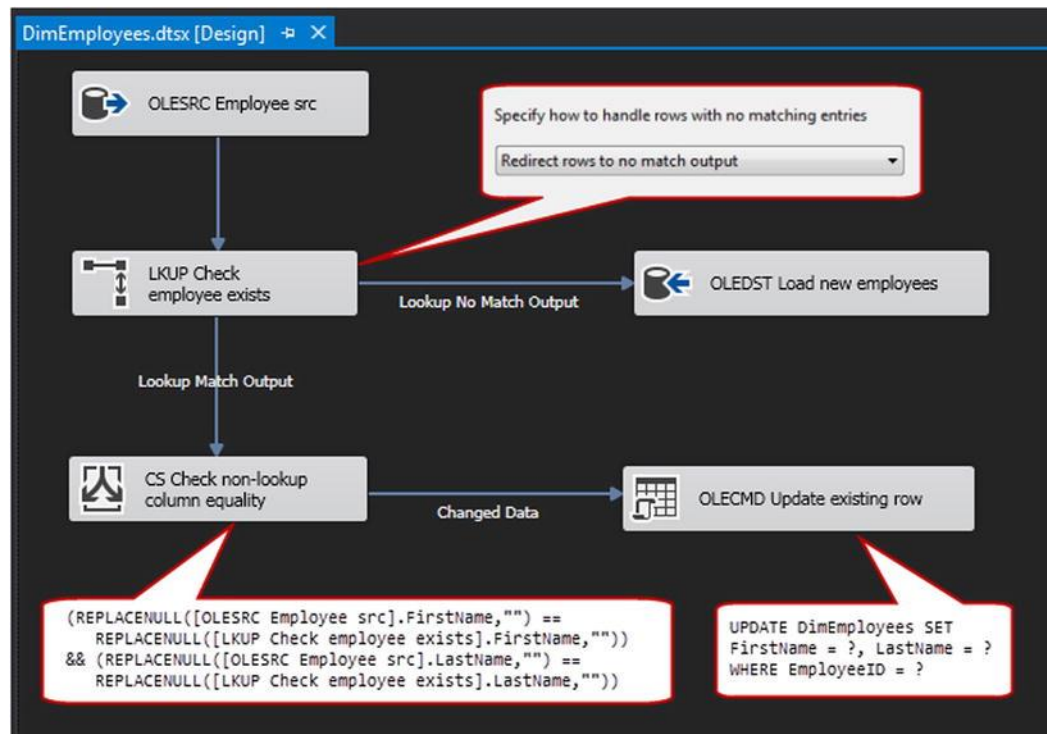
Route changed rows to the target table.

Unmatched rows from the previous step can then be routed directly to the target table using the appropriate database destination component.

For the source rows that have a business key match in the target table, compare the other values that may change.

If any of those source values differs from the value found for that row in the destination table, those rows in the destination table will be updated with the new values.

Incremental Loads in SSIS



Incremental Loads in SSIS

Delta load using Merge Join

The logic:

If there are any New records in Source data, then we have to insert those records in the target table.

If there are any updated values in Source data, then we have to update those records in the target table.

If there are any deleted values in Source data, it is recommended to keep them in the target table and precise the deletion date.

SSIS:

Define source to staging data and the currently available source data

Use the MERGE JOIN component to make a join and merge staging version of the data with current source version of the data.

Depending on the result perform proper split of the rows (new, updated and deleted) and apply proper modifications to each type

Use additional columns – InsertionDate, UpdateDate, and DeletionDate

Incremental Loads in SSIS

After the Merge Join, you can use the Conditional Split Component to treat these 3 conditions.

InsertNewRecords:

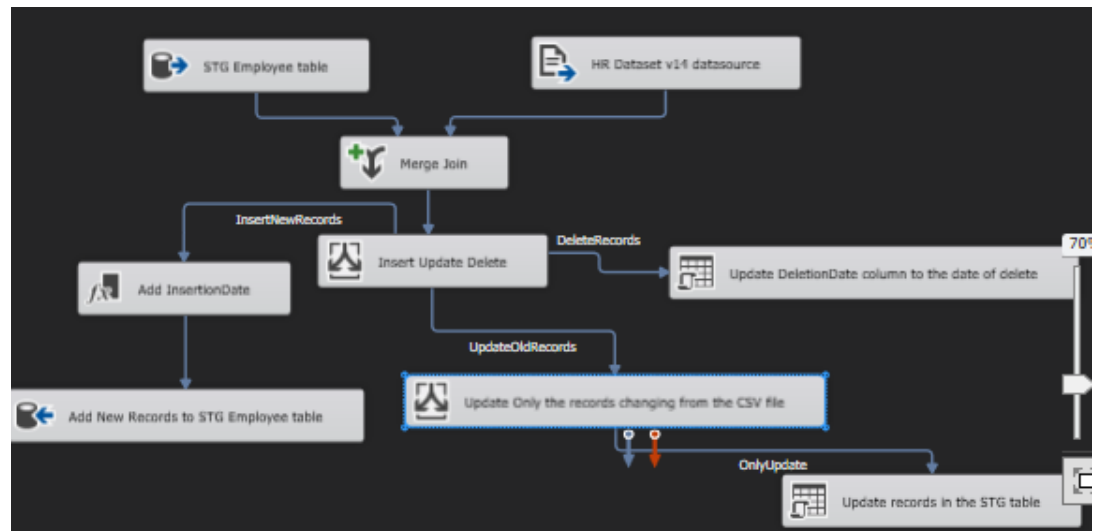
*ISNULL(EmpIDSTG) &&
!ISNULL(EmpIDSRC)*

UpdateOldRecords:

*!ISNULL(EmpIDSTG) &&
!ISNULL(EmpIDSRC)*

DeleteRecords:

*!ISNULL(EmpIDSTG) &&
ISNULL(EmpIDSRC)*



Incremental Loads in SSIS

For new rows

- Insert into staging table

 - Apply proper insertionDate*

For updated rows

- Run a OLE DB Command to update the rows

 - Apply proper updateDate*

For deleted rows

- Do a soft delete

 - Apply proper deletionDate*

Incremental Loads in SSIS

Another approach is to use the Slowly Changing Dimension (SCD) Wizard

As its name implies, it is designed specifically for managing SCD elements in a data warehouse.

However, its use is certainly not limited to dimension processing.

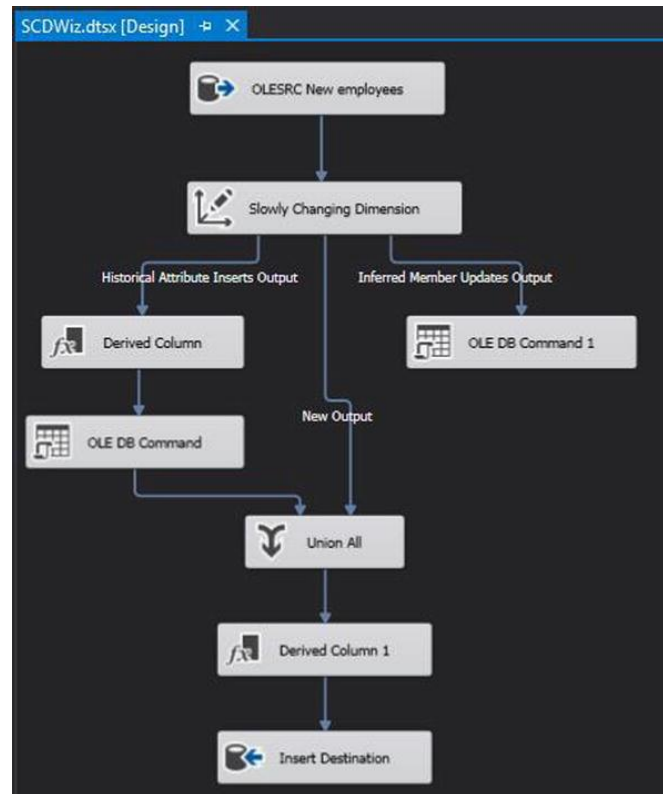
the wizard guides you through the configuration, including the following:

Which columns should be engaged as part of the SCD, along with the option to handle changes as simple updates (Type 1) or historical values (Type 2).

How to identify current vs. expired rows, if any Type 2 columns are present; you can specify either a flag or a date span to indicate currency of SCD records.

How to handle inferred members (discussed in more depth shortly).

Incremental Loads in SSIS



Incremental Loads in SSIS

However, the SCD Wizard does have some significant drawbacks.

Performance:

The wizard performs reasonably well against small sets of data.

However, because many of the operations are performed on a row-by-row basis, leveraging this transform against sizeable dimensions can cause a significant performance bottleneck.

Destructive changes:

When you run the wizard, all of the required transforms and destinations are created automatically.

Similarly, if you reconfigure the SCD transform (for example, changing a column type from a Type 1 to a Type 2 historical), the existing design elements are removed and added back to the data flow.

As a result, any changes that you've made to that data path will be lost if you make a change to the SCD transform.

No direct support for auditing:

Although you can add your own auditing logic, it's not a native function of this component.

As a result of these shortcomings (especially the performance implications), the SCD Wizard doesn't see a lot of action in the real world.

Incremental Loads in SSIS

Another approach is to use MERGE statement in T-SQL

The anatomy of a MERGE statement looks something like this:

Specify the source data.

Specify the target table.

Choose the columns on which to join the source and target data.

Indicate the column alignment between the two sets of data to be used for determining whether matched records are different in some way.

Define the logic for cases when the data is changed, or if it only exists in either the source or the destination.

There are no native hooks to the MERGE statement in Integration Services

the implementation of MERGE in an SSIS package is done through an Execute SQL task.

Incremental Loads in SSIS

As part of an SCD MERGE upsert process, our SSIS package would contain tasks to perform the following functions:

- Remove previously staged data from the staging table.

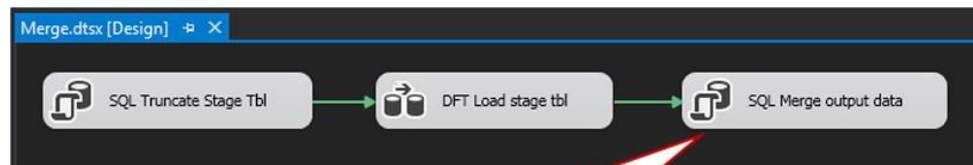
- Load the staging table from the source system.

- Clean up the staged data (if required).

- Execute the MERGE statement to upsert the staged data to the dimension table.

- Log the upsert operation (optional).

Incremental Loads in SSIS



```
MERGE DimEmployees as target                                -- Destination table
USING (SELECT * FROM src.Employees) AS source                -- Staging (source) table
ON target.SSN = source.SSN                                   -- Match criteria
WHEN MATCHED AND NOT (
    ISNULL(source.FirstName, '') = ISNULL(target.FirstName, '')
    AND ISNULL(source.LastName, '') = ISNULL(target.LastName, '')
    AND ISNULL(source.Gender, '') = ISNULL(target.Gender, '')
    AND ISNULL(source.BirthDate, '1/1/1900') = ISNULL(target.BirthDate, '1/1/1900')
    AND ISNULL(source.EmpStatus, '1/1/1900') = ISNULL(target.EmpStatus, '1/1/1900')
)
THEN UPDATE                                                  -- Same person but different details
SET target.FirstName = source.FirstName
, target.LastName = source.LastName
, target.Gender = source.Gender
, target.BirthDate = source.BirthDate
, target.EmpStatus = source.EmpStatus

WHEN NOT MATCHED BY target -- Person does not yet exist in target table :: Insert
THEN INSERT (SSN, FirstName, LastName, Gender, BirthDate, EmpStatus)
VALUES (source.SSN, source.FirstName, source.LastName, source.Gender
, source.BirthDate, source.EmpStatus
)

OUTPUT $action, inserted.*;
```

Incremental Loads in SSIS

Another significant incremental load feature is the change data capture (CDC)

CDC is a feature of the database engine that allows the collection of data changes to monitored tables.

CDC is a supply-side incremental load tool that is enabled first at the database level and then implemented on a table-by-table basis via capture instances

Once CDC is enabled for a table, the database engine uses the transaction log to track all DML operations (INSERTs, UPDATEs, and DELETEs), logging each change to the change table for each monitored table.

The change table contains not only the fact that there was a change to the data, but it also maintains the history of those changes.

CDC in Integration Services

SSIS can consume CDC using common native SSIS components, you can access the change table to capture the data changes.

Handling errors

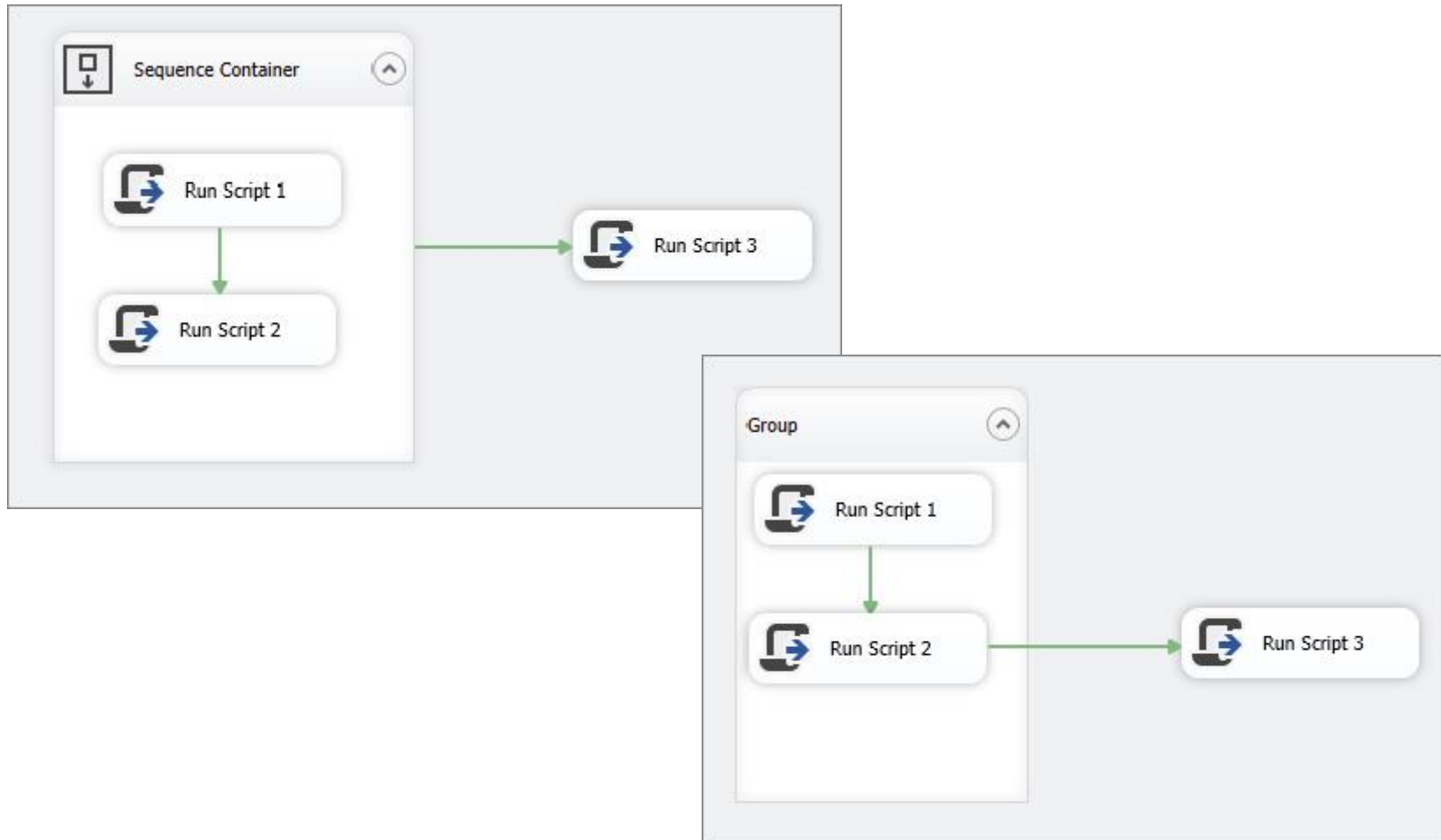
Fail Package on Error

There are situations where the data warehouse ETL process should be designed to fail if error conditions arise.

It's worth noting here that to properly use this design pattern, you must include some logic to roll back changes due to a partial load.



Containers / groups



Looping

The For Loop Container enables you to create looping in your package similar to how you would loop in nearly any programming language.

In this looping style, SSIS optionally initializes an expression and continues to evaluate it until the expression evaluates to false.

a Message Queue

The Foreach Loop Container is a powerful looping mechanism that enables you to loop through a collection of objects.

As you loop through the collection, the container assigns the value from the collection to a variable, which can later be used by tasks or connections inside or outside the container.

Files, items, variable, tables, rows, schema, node list in XML, etc.

Fuzzy transformations

Fuzzy Lookup and Fuzzy Grouping, deal with associating data through data similarity, rather than exact data matching.

The “fuzzy” part of the transformation name refers to data coupling based on selected data mapping using defined similarity and confidence measurements.

Here is a brief description of each:

Fuzzy Lookup:

The Fuzzy Lookup Transformation takes input data from a Data Flow and matches it to a specified table within SQL Server joined across data similarity column matching.

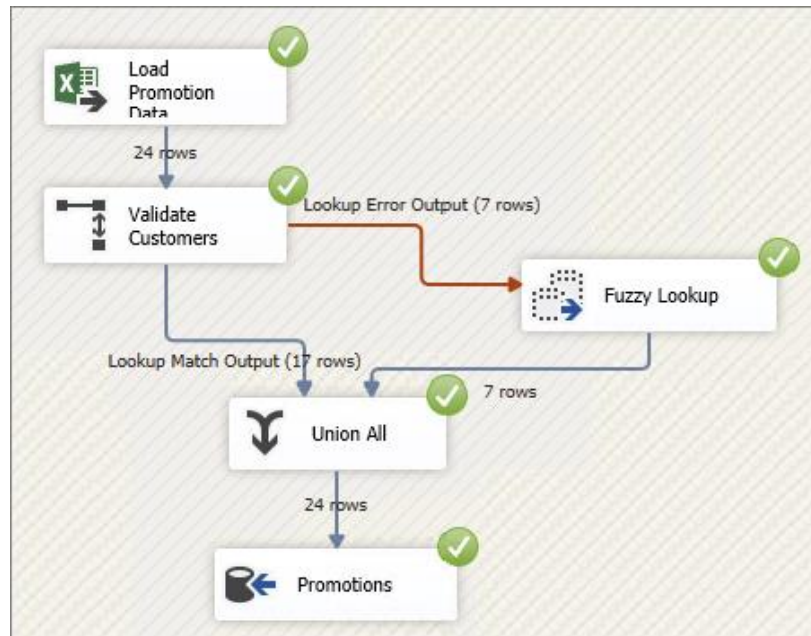
The Fuzzy Lookup is like the Lookup Transformation, except that the column mapping can be adjusted to evaluate data likeness and the output can be tuned to return one or more potential results.

Fuzzy Grouping:

This transformation takes a single input from the Data Flow and performs a comparison with itself to try to identify potential duplicates in the data.

The grouping doesn't evaluate all the columns in the source input; it only searches for duplicates across the columns you select based on the similarity settings that you define.

Fuzzy transformations



Some additional notes

Remember to use:

- naming patterns
- annotations

Plenty of online resources

- Look for SSIS patterns

Kimball R., Ross M.

The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd Edition

Wiley Publishing 2013

Cote C., Lah M., Sarka D.

SQL Server 2017 Integration Services Cookbook

Apress, 2017

Knight B., Knight D., Moss J.M., Davis M., Rock C.

Professional Microsoft SQL Server 2014 Integration Services

Apress, 2014

Leonard A., Mitchell T., Masson M., Moss J., Ufford M.

SQL Server Integration Services Design Patterns, Second Edition

Apress, 2014

Bibliography

SOURCES